

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра вычислительных систем

Отчет по лабораторной работе №3
по дисциплине
Архитектура распределенных приложений

по теме:
KUBERNETES.DEPLOY ПРИЛОЖЕНИЕ НА СЕРВЕР

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватель:
Преподаватель

Т.В Челканова

Новосибирск 2025 г.

СОДЕРЖАНИЕ

ЦЕЛЬ И ЗАДАЧИ	3
0.1 Цель	3
0.2 Задание.....	3
1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....	4
1.1 Что такое Kubernetes?	4
1.2 Структура сервера	4
1.2.1 Cluster	4
1.2.2 Node	4
1.2.3 Master	4
1.2.4 etcd.....	5
1.2.5 Controller Manager	5
1.2.6 Scheduler	5
1.2.7 Worker	5
1.2.8 Container	5
1.2.9 API.....	6
1.2.10 Pod.....	6
1.3 Kubernetes API	6
1.4 Kubernetes manifest	6
1.4.1 Service	7
1.4.2 Deployment	7
1.4.3 Ingress	7
2 ХОД РАБОТЫ	8
2.1 Доработка GITLAB CI-CD.....	8
2.2 Разработка deployment.yaml	9
2.3 Разработка service.yaml	10
2.4 Разработка ingress.yaml	11
3 ВЫВОД	12

ЦЕЛЬ И ЗАДАЧИ

0.1 Цель

Познакомиться с Kubernetes. Сделать Deploy приложения на сервер

0.2 Задание

:

1. Познакомиться с Kubernetes.
2. Настроить Kubernetes.
3. Доработать CI/CD, таким образом, чтобы осуществлялся автоматический деплой приложения на сервер.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Что такое Kubernetes?

Kubernetes — это открытая система для автоматизации развертывания, масштабирования и управления контейнеризированными приложениями. Она обеспечивает высокую доступность, отказоустойчивость и динамическую оркестрацию контейнеров в кластере.

1.2 Структура сервера

1.2.1 Cluster

Кластер — это группа объединённых нод (мастеров и воркеров), работающих вместе как единая система. Кластер обеспечивает высокую доступность, масштабируемость и распределённое выполнение приложений.

1.2.2 Node

Нода — это физический или виртуальный сервер, являющийся частью кластера Kubernetes. Нода может быть либо мастер-нодой, либо воркер-нодой. На нодах запускаются системные компоненты и контейнеры приложений.

1.2.3 Master

Мастер-сервер - это основной управляющий компонент распределённой системы (например, кластера Kubernetes), который принимает решения о размещении рабочих нагрузок, контролирует состояние кластера, управляет планировщиком и обеспечивает работу основных управляющих служб. Он координирует работу всех остальных узлов.

1.2.4 etcd

etcd - это распределённое, надёжное хранилище ключ–значение, используемое Kubernetes для сохранения конфигурации кластера, его текущего состояния и метаданных. Оно обеспечивает согласованность данных между всеми компонентами кластера и является критически важным элементом управления состоянием.

1.2.5 Controller Manager

Controller Manager — это компонент мастер-ноды Kubernetes, который запускает и координирует работу контроллеров. Каждый контроллер отвечает за поддержание определённого аспекта желаемого состояния кластера: репликации подов, обновления нод, управления конечными точками (endpoints) и многого другого. Controller Manager следит за изменениями и автоматически предпринимает действия для достижения требуемого состояния.

1.2.6 Scheduler

Scheduler — компонент мастер-ноды Kubernetes, который анализирует требования подов и доступные ресурсы нод, выбирая оптимальную ноду для размещения каждого пода.

1.2.7 Worker

Воркер - это рабочий узел кластера, на котором фактически запускаются приложения и контейнеры. Каждый воркер предоставляет вычислительные ресурсы (CPU, память, сеть) и исполняет рабочие задачи, переданные ему мастер-сервером.

1.2.8 Container

Контейнер — это изолированная среда выполнения приложения, содержащая всё необходимое: библиотеки, зависимости и конфигурацию. Kubernetes управляет контейнерами, объединяя их в поды.

1.2.9 API

Kubernetes API Server — это центральный интерфейс взаимодействия всех компонентов кластера. Он принимает запросы от пользователей, kubectl, контроллеров и других частей системы, валидирует их и сохраняет изменения в etcd. API Server представляет собой основной коммуникационный слой Kubernetes и обеспечивает доступ к объектам кластера через REST API.

1.2.10 Pod

Под — это минимальная единица развертывания в Kubernetes. Он содержит один или несколько контейнеров, разделяющих сеть и файловую систему. Все контейнеры в поде запускаются на одной ноде и работают как единое логическое приложение.

1.3 Kubernetes API

kubectl — это командная утилита для взаимодействия с API Kubernetes. С помощью kubectl можно:

```
// apply manifest
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f ingress.yaml

//show deployments list in namespace ia331
kubectl get deployments -n ia331
//show pods list in namespace ia331
kubectl get pods -n ia331
//show service list in namespace ia331
kubectl get svc -n ia331
//show ingress list in namespace ia331
kubectl get ingress -n ia331
```

1.4 Kubernetes manifest

В Kubernetes манифест - это файл (обычно в формате YAML или JSON), который описывает желаемое состояние объекта кластера.

1.4.1 Service

Манифест Service определяет объект, который обеспечивает сетевой доступ к подам. Он создаёт стабильную точку входа, распределяет нагрузку и скрывает изменения подов.

1.4.2 Deployment

Манифест Deployment описывает, как должны запускаться и обновляться поды: количество реплик, контейнерный образ, стратегию обновления. Обеспечивает автоматическое восстановление и масштабирование.

1.4.3 Ingress

Манифест Ingress задаёт правила внешнего HTTP/HTTPS-доступа к сервисам внутри кластера. Он управляет маршрутизацией по доменам и путям, SSL-терминацией и связан с Ingress Controller.

ХОД РАБОТЫ

2.1 Доработка GITLAB CI-CD

```
stages:
  - build
  - deploy

build:
  stage: build
  image: docker:24
  services:
    - name: docker:24-dind
  before_script:
    - echo "$HAWK_REGISTRY_PASSWORD" | docker login -u
      "$HAWK_REGISTRY_USER" --password-stdin "$HAWK_REGISTRY"
    - docker context create context
    - docker buildx create --name mybuilder --driver
      docker-container --use context
  script:
    - TAGS="$HAWK_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA"
    - if [ -n "$CI_COMMIT_TAG" ]; then TAGS="$TAGS -t $
      HAWK_REGISTRY_IMAGE:$CI_COMMIT_TAG"; fi
    - docker buildx build --platform linux/amd64,linux/arm64 -f
      ./Dockerfile ./ -t $TAGS --push
    - echo "TAGS=${TAGS}" > tags.env
  rules:
    - if: "$CI_COMMIT_TAG"
      when: always
    - when: manual

artifacts:
  reports:
    dotenv: tags.env

deploy:
  stage: deploy
  image: alpine:latest
  before_script:
    - apk add --no-cache curl gettext
```



```

- curl -LO
  "https://dl.k8s.io/release/v1.30.7/bin/linux/arm64/kubectl"
- install -m 0755 kubectl /usr/local/bin/kubectl
script:
- echo "Deploy with tag $TAGS"
- ls -la
- envsubst < deployment.yaml | kubectl apply -f -
- kubectl apply -f service.yaml
- kubectl apply -f ingress.yaml

rules:
- when: manual

```

Добавим джобу deploy, в которой с помощью envsubst обновлять image в deployment.yaml. Это позволит видеть изменения в приложении после push в репозиторий.

2.2 Разработка deployment.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: ia331s08-app
  namespace: ia331
  labels:
    owner: ia331s08
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ia331s08-app
  template:
    metadata:
      labels:
        app: ia331s08-app
        owner: ia331s08
    spec:
      serviceAccountName: ci
      containers:
        - name: ia331s08-app
          image: ${TAGS}

```

```

ports:
  - containerPort: 8000
env:
  - name: PORT
    value: "8000"
securityContext:
  allowPrivilegeEscalation: false
resources:
  requests:
    memory: "32Mi"
    cpu: "32m"
  limits:
    memory: "64Mi"
    cpu: "64m"
readinessProbe:
  httpGet:
    path: /
    port: 8000
  initialDelaySeconds: 5
  periodSeconds: 10
livenessProbe:
  httpGet:
    path: /
    port: 8000
  initialDelaySeconds: 15
  periodSeconds: 20

```

2.3 Разработка service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: ia331s08-app-svc
  namespace: ia331
  labels:
    owner: ia331s08
spec:
  selector:
    app: ia331s08-app
  ports:

```

```
- port: 80
  targetPort: 8000
```

2.4 Разработка ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ia331s08-app-ingress
  namespace: ia331
  labels:
    owner: ia331s08
spec:
  ingressClassName: traefik-dev
  rules:
    - host: ia331s08-app.hawk-dev.csc.sibsutis.ru
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: ia331s08-app-svc
                port:
                  number: 80
```

ВЫВОД

В ходе работы я познакомился с Kubernetes - важным инструментом в разработке. Настроил Kubernetes на кластере и задеплоил свое приложение на сервер.